

Linear Search & Binary Search

Data Structures & Algorithms (DSA) — Study Notes

1. Searching in DSA

Searching means finding a required element (target/key) in a collection of data. Two basic techniques are Linear Search and Binary Search.

2. Linear Search

Definition: Linear Search checks elements one by one until the target is found or all elements have been checked.

Use it when: the array may be unsorted, the data is small, or simplicity is important.

Algorithm

1. Start from the first element.
2. Compare it with the target.
3. If equal, return its index.
4. Otherwise move to the next element.
5. If the end is reached, return -1.

Example

Array: [10, 25, 7, 42, 18], Target: 42

Checks: 10 → 25 → 7 → 42 → **Found**

C++ Implementation

```
int linearSearch(int arr[], int n, int target) {  
    for (int i = 0; i < n; i++) {  
        if (arr[i] == target)  
            return i;  
    }  
    return -1;  
}
```

Complexity

Case	Time
Best	$O(1)$
Average	$O(n)$
Worst	$O(n)$
Space	$O(1)$

3. Binary Search

Definition: Binary Search repeatedly divides a **sorted** array into two halves and decides which half can contain the target.

Important: The standard binary-search algorithm requires sorted data.

Algorithm

1. Set low = 0 and high = n - 1.
2. Calculate mid = low + (high - low) / 2.
3. If arr[mid] equals the target, return mid.
4. If arr[mid] is smaller, search the right half.
5. If arr[mid] is larger, search the left half.
6. Stop when low > high; then the target is absent.

Example

Sorted array: [5, 10, 15, 20, 25, 30, 35], Target: 30

Middle 20 → target is larger → search right half → middle 30 → **Found**.

C++ Implementation (Iterative)

```
int binarySearch(int arr[], int n, int target) {
    int low = 0, high = n - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (arr[mid] == target)
            return mid;
        else if (arr[mid] < target)
            low = mid + 1;
        else
            high = mid - 1;
    }

    return -1;
}
```

Complexity

Case	Time
Best	O(1)
Average	O(log n)
Worst	O(log n)
Space (iterative)	O(1)

4. Linear Search vs Binary Search

Feature	Linear Search	Binary Search
Data	Sorted or unsorted	Must be sorted
Approach	One by one	Divide in half
Best	O(1)	O(1)
Average	O(n)	O(log n)

Worst	$O(n)$	$O(\log n)$
Extra space	$O(1)$	$O(1)$ iterative

5. Quick Exam Tips

- **Linear Search:** check elements one by one.
- **Binary Search:** sorted array + middle element + eliminate half.
- Binary Search is usually much faster for large sorted arrays.
- Use **$\text{mid} = \text{low} + (\text{high} - \text{low}) / 2$** in C++.
- Both sample functions return **-1** when the target is not found.

Summary

Use Linear Search for unsorted or small data. Use Binary Search when the data is sorted and fast searching is needed.